Fix, Fork, Contribute
Thank you jiixing
for 2 contributions

WebGPU Timing Performance

Let's go over various things you might want to time for performance. We'll time 3 things:

- The frame rate in frames per second (fps)
- The time spent in JavaScript per frame
- The time spent on the GPU per frame

First, let's take a circle example from [the article on vertex buffers] and lets animate them so we have something that's easy to see changes in how much time things take.

In that example we had 3 vertex buffers. One was for the positions and brightness of the vertices for a circle. One was for things that are per instance but static which included the circle's offset and color. And, the last one was for things that change each time we render, in this case it was the scale so we could keep the aspect ratio of the circles correct so they stayed circles and not ellipses as the user changed the size of the window.

We want to animate them moving so let's move the offset to the same buffer as the scale. First we'll change the render pipeline to move the offset to the same buffer as the scale.

hide deleted

```
const pipeline = device.createRenderPipeline({
  label: 'per vertex color',
  layout: 'auto',
  vertex: {
    module,
    buffers: [
      {
        arrayStride: 2 * 4 + 4, // 2 floats, 4 bytes each + 4 bytes
        attributes: [
          {shaderLocation: 0, offset: 0, format: 'float32x2'},  // position
          {shaderLocation: 4, offset: 8, format: 'unorm8x4'},   // perVertexColor
        ],
      },
      {
        arrayStride: 4 + 2 * 4, // 4 bytes + 2 floats, 4 bytes each
        arrayStride: 4, // 4 bytes
        stepMode: 'instance',
        attributes: [
          {shaderLocation: 1, offset: 0, format: 'unorm8x4'},   // color
          {shaderLocation: 2, offset: 4, format: 'float32x2'},  // offset
        ],
```

```
      },
      {
        arrayStride: 2 * 4, // 2 floats, 4 bytes each
        arrayStride: 4 * 4, // 4 floats, 4 bytes each
        stepMode: 'instance',
        attributes: [
          {shaderLocation: 3, offset: 0, format: 'float32x2'},   // scale
          {shaderLocation: 2, offset: 0, format: 'float32x2'},  // offset
          {shaderLocation: 3, offset: 0, format: 'float32x2'},   // scale
          {shaderLocation: 3, offset: 8, format: 'float32x2'},   // scale
        ],
      },
    ],
  },
  fragment: {
    module,
    targets: [{ format: presentationFormat }],
  },
});
```

Then we'll change the part that sets up the vertex buffers to move the offsets together with the scales.

hide deleted

```
// create 2 vertex buffers
const staticUnitSize =
  4 +      // color is 4 bytes
  2 * 4;  // offset is 2 32bit floats (4bytes each)
  4;     // color is 4 bytes
const changingUnitSize =
  2 * 4;  // scale is 2 32bit floats (4bytes each)
  2 * 4 + // offset is 2 32bit floats (4bytes each)
  2 * 4;  // scale is 2 32bit floats (4bytes each)
const staticVertexBufferSize = staticUnitSize * kNumObjects;
const changingVertexBufferSize = changingUnitSize * kNumObjects;

const staticVertexBuffer = device.createBuffer({
  label: 'static vertex for objects',
  size: staticVertexBufferSize,
  usage: GPUBufferUsage.VERTEX | GPUBufferUsage.COPY_DST,
});

const changingVertexBuffer = device.createBuffer({
  label: 'changing storage for objects',
  size: changingVertexBufferSize,
  usage: GPUBufferUsage.VERTEX | GPUBufferUsage.COPY_DST,
});

// offsets to the various uniform values in float32 indices
const kColorOffset = 0;
const kOffsetOffset = 1;

const kScaleOffset = 0;
const kOffsetOffset = 0;
const kScaleOffset = 2;

{
  const staticVertexValuesU8 = new Uint8Array(staticVertexBufferSize);
  const staticVertexValuesF32 = new Float32Array(staticVertexValuesU8.buffer);
  for (let i = 0; i < kNumObjects; ++i) {
    const staticOffsetU8 = i * staticUnitSize;
    const staticOffsetF32 = staticOffsetU8 / 4;
```

```
      // These are only set once so set them now
      staticVertexValuesU8.set(        // set the color
          [rand() * 255, rand() * 255, rand() * 255, 255],
          staticOffsetU8 + kColorOffset);

      staticVertexValuesF32.set(       // set the offset
          [rand(-0.9, 0.9), rand(-0.9, 0.9)],
          staticOffsetF32 + kOffsetOffset);

      objectInfos.push({
        scale: rand(0.2, 0.5),
        offset: [rand(-0.9, 0.9), rand(-0.9, 0.9)],
        velocity: [rand(-0.1, 0.1), rand(-0.1, 0.1)],
      });
    }
    device.queue.writeBuffer(staticVertexBuffer, 0, staticVertexValuesF32);
    device.queue.writeBuffer(staticVertexBuffer, 0, staticVertexValuesU8);
  }
```

At render time we can update the offsets of the circles based on their velocity and then upload those to the GPU.

hide deleted

```
  const euclideanModulo = (x, a) => x - a * Math.floor(x / a);

  let then = 0;
  function render() {
  function render(now) {
    now *= 0.001;  // convert to seconds
    const deltaTime = now - then;
    then = now;

...
      // set the scales for each object
    objectInfos.forEach(({scale}, ndx) => {
      const offset = ndx * (changingUnitSize / 4);
      vertexValues.set([scale / aspect, scale], offset + kScaleOffset); // set the scale
    objectInfos.forEach(({scale, offset, veloctiy}, ndx) => {
      // -1.5 to 1.5
      offset[0] = euclideanModulo(offset[0] + velocity[0] * deltaTime + 1.5, 3) - 1.5;
      offset[1] = euclideanModulo(offset[1] + velocity[1] * deltaTime + 1.5, 3) - 1.5;

      const off = ndx * (changingUnitSize / 4);
      vertexValues.set(offset, off + kOffsetOffset);
      vertexValues.set([scale / aspect, scale], off + kScaleOffset);
    });

...

    requestAnimationFrame(render);
  }
  requestAnimationFrame(render);

  const observer = new ResizeObserver(entries => {
    for (const entry of entries) {
      const canvas = entry.target;
      const width = entry.contentBoxSize[0].inlineSize;
      const height = entry.contentBoxSize[0].blockSize;
      canvas.width = Math.max(1, Math.min(width, device.limits.maxTextureDimension2D));
      canvas.height = Math.max(1, Math.min(height, device.limits.maxTextureDimension2D));
      // re-render
```

```
      render();
    }
  });
  observer.observe(canvas);
```

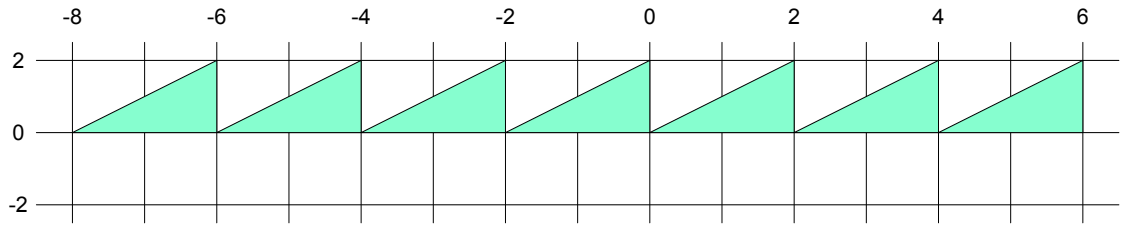
We also switched to a rAF loop[1].

The code above uses `euclideanModulo` to update the offset. `euclideanModulo` returns the remainder of a division where the remainder always is always positive and in the same direction. For example

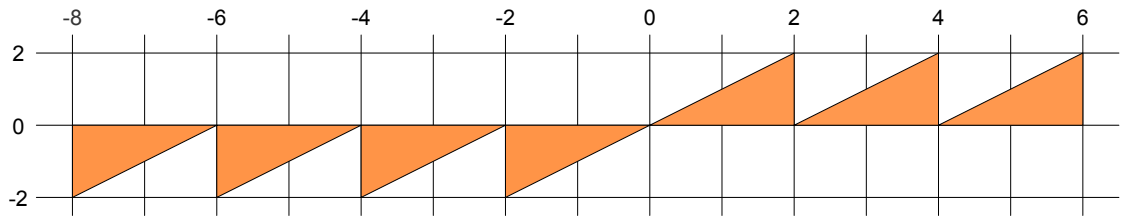
value	% operator	euclideanModulo
0.3	0.3	0.3
2.3	0.3	0.3
4.3	0.3	0.3
-1.7	-1.7	0.3
-3.7	-1.7	0.3

modulo 2 of % vs euclideanModulo

To put it another way, here's a graph of the `%` operator vs `euclideanModulo`



euclideanModule(v, 2)



v % 2

So, the code above takes the offset, which is in clip space, and adds 1.5. It then takes the euclideanModulo by 3 which will give us a number that is wrapped between 0.0 and 3.0 and then subtracts 1.5. This gives us numbers that stay between -1.5 and +1.5 and lets them wrap around to the other side. We use -1.5 to +1.5 so that the circles don't wrap until they are off the screen. [2]

To give us something to adjust, lets make it so we can set how many circles to draw.

hide deleted

```
- const kNumObjects = 100;
  const kNumObjects = 10000;


...

  const settings = {
    numObjects: 100,
  };

  const gui = new GUI();
  gui.add(settings, 'numObjects', 0, kNumObjects, 1);

  ...

    // set the scale and offset for each object
- objectInfos.forEach(({scale, offset, velocity}, ndx) => {
    for (let ndx = 0; ndx < settings.numObjects; ++ndx) {
      const {scale, offset, velocity} = objectInfos[ndx];

      // -1.5 to 1.5
      offset[0] = euclideanModulo(offset[0] + velocity[0] * deltaTime + 1.5, 3) - 1.5;
      offset[1] = euclideanModulo(offset[1] + velocity[1] * deltaTime + 1.5, 3) - 1.5;

      const off = ndx * (changingUnitSize / 4);
      vertexValues.set(offset, off + kOffsetOffset);
      vertexValues.set([scale / aspect, scale], off + kScaleOffset);
- });
    }

    // upload all offsets and scales at once
- device.queue.writeBuffer(changingVertexBuffer, 0, vertexValues);
    device.queue.writeBuffer(
        changingVertexBuffer, 0,
        vertexValues, 0, settings.numObjects * changingUnitSize / 4);

- pass.draw(numVertices, kNumObjects);
    pass.draw(numVertices, settings.numObjects);
```

So now we should have something that animates and we can adjust how much work is done by setting the number of circles.

click here to open in a separate window

To that, let's add frames per second (fps) and time spent in JavaScript

First we need a way to display this info so lets add an `<pre>` element positioned on top of the canvas.

```
<body>
  <canvas></canvas>
  <pre id="info"></pre>
</body>
```

```
html, body {
  margin: 0;       /* remove the default margin          */
  height: 100%;    /* make the html,body fill the page   */
}
canvas {
  display: block;  /* make the canvas act like a block   */
  width: 100%;     /* make the canvas fill its container */
  height: 100%;
}
#info {
  position: absolute;
  top: 0;
  left: 0;
  margin: 0;
  padding: 0.5em;
  background-color: rgba(0, 0, 0, 0.8);
  color: white;
}
```

We already have the data needed to display frames per second. It's the `deltaTime` we computed above.

For JavaScript time, we can record the time our `requestAnimationFrame` started and the time it ended.

```
let then = 0;
function render(now) {
  now *= 0.001;  // convert to seconds
  const deltaTime = now - then;
  then = now;

  const startTime = performance.now();

  ...

  const jsTime = performance.now() - startTime;

  infoElem.textContent = `\
fps: ${(1 / deltaTime).toFixed(1)}
js: ${jsTime.toFixed(1)}ms
`;

  requestAnimationFrame(render);
}
requestAnimationFrame(render);
```

And that gives us our first two timing measurements.

click here to open in a separate window

Timing the GPU

WebGPU provides an **optional** `'timestamp-query'` feature for checking how long an operation takes on the GPU. Since it's an optional feature we need to see if it exists and request it like we covered in the article on limits and features.

hide deleted

```
async function main() {
  const adapter = await navigator.gpu?.requestAdapter();
  const device = await adapter?.requestDevice();
  const canTimestamp = adapter.features.has('timestamp-query');
  const device = await adapter?.requestDevice({
    requiredFeatures: [
      ...(canTimestamp ? ['timestamp-query'] : []),
    ],
  });
  if (!device) {
    fail('need a browser that supports WebGPU');
    return;
  }
```

Above, we set `canTimestamp` to true or false based on if the adapter supports the `'timestamp-query'` feature. If it does we require that feature when we create our device.

With the feature enabled we can ask WebGPU for *timestamps* for a render pass or compute pass. You do this by making a `GPUQuerySet` and adding it to your compute or render pass. A `GPUQuerySet` is effectively an array of query results. You tell WebGPU which element in the array to record the time the pass started and which element in the array to record when the pass ended. You can then copy those timestamps to a buffer and map the buffer to read the results.[3]

So, first we create a query set.

```
const querySet = device.createQuerySet({
   type: 'timestamp',
   count: 2,
});
```

We need count to be at least 2 so we can write both a start and end timestamp.

We need a buffer to convert the querySet info into data we can access.

```
const resolveBuffer = device.createBuffer({
  size: querySet.count * 8,
  usage: GPUBufferUsage.QUERY_RESOLVE | GPUBufferUsage.COPY_SRC,
});
```

Each element in a querySet takes 8 bytes. We need to give it a usage of `QUERY_RESOLVE` and, if we want be able to read the results back in JavaScript we need the `COPY_SRC` usage so we can copy the result to a mappable buffer.

Finally we create a mappable buffer to read the results.

```
const resultBuffer = device.createBuffer({
  size: resolveBuffer.size,
  usage: GPUBufferUsage.COPY_DST | GPUBufferUsage.MAP_READ,
});
```

We need to wrap this code in a way that only creates these things if the feature exists, otherwise we'll get an error trying to make a `'timestamp'` querySet.

```
const { querySet, resolveBuffer, resultBuffer } = (() => {
  if (!canTimestamp) {
    return {};
  }

  const querySet = device.createQuerySet({
     type: 'timestamp',
     count: 2,
  });
  const resolveBuffer = device.createBuffer({
    size: querySet.count * 8,
    usage: GPUBufferUsage.QUERY_RESOLVE | GPUBufferUsage.COPY_SRC,
  });
  const resultBuffer = device.createBuffer({
    size: resolveBuffer.size,
    usage: GPUBufferUsage.COPY_DST | GPUBufferUsage.MAP_READ,
  });
  return {querySet, resolveBuffer, resultBuffer };
})();
```

In our render pass descriptor we tell it the querySet to use and the index of the elements in the querySet to write the start and ending timestamps.

```
const renderPassDescriptor = {
  label: 'our basic canvas renderPass with timing',
  colorAttachments: [
    {
      // view: <- to be filled out when we render
      clearValue: [0.3, 0.3, 0.3, 1],
      loadOp: 'clear',
      storeOp: 'store',
    },
  ],
  ...(canTimestamp && {
    timestampWrites: {
      querySet,
      beginningOfPassWriteIndex: 0,
      endOfPassWriteIndex: 1,
    },
```

```
  },
};
```

Above, if the feature exists, we add a `timestampWrites` section to our renderPassDescriptor and pass in the querySet and tell it to write the start to element 0 of the set and the end to element 1.

After we end the pass, we need to call `resolveQuerySet`. This takes the results of the query and puts them in a buffer. We pass it the querySet, the first index in the query set where to start resolving, the number of entries to resolve, a buffer to resolve to, and an offset in that buffer where to store the result.

```
pass.end();

if (canTimestamp) {
  encoder.resolveQuerySet(querySet, 0, querySet.count, resolveBuffer, 0);
}
```

We also want to copy the `resolveBuffer` to our `resultsBuffer` so we can map it and look at the results in JavaScript. We have an issue though. We can not copy to our `resultsBuffer` while it's mapped. Fortunately buffers have a `mapState` property we can check. If it's set to `unmapped`, the value it starts with, then it's safe to copy to it. Other values are `'pending'`, the value it becomes the moment we call `mapAsync`, and `'mapped'`, the value it is when `mapAsync` resolves. After we `unmap` it it goes back to `'unmapped'`.

```
if (canTimestamp) {
  encoder.resolveQuerySet(querySet, 0, 2, resolveBuffer, 0);
  if (resultBuffer.mapState === 'unmapped') {
    encoder.copyBufferToBuffer(resolveBuffer, 0, resultBuffer, 0, resultBuffer.size);
  }
}
```

After we've submitted the command buffer we can map the `resultBuffer`. Like above, only want to map it if it's `'unmapped'`.

```
let gpuTime = 0;

 ...

 function render(now) {

  ...

  const commandBuffer = encoder.finish();
  device.queue.submit([commandBuffer]);
```

```
    if (canTimestamp && resultBuffer.mapState === 'unmapped') {
      resultBuffer.mapAsync(GPUMapMode.READ).then(() => {
        const times = new BigInt64Array(resultBuffer.getMappedRange());
        gpuTime = Number(times[1] - times[0]);
        resultBuffer.unmap();
      });
    }
```

Query set results are in nanoseconds and are stored in 64bit integers. To read them in JavaScript we can use a `BigInt64Array` typedarray view. Using `BigInt64Array` requires special care. When you read an element from a `BitInt64Array` the type is a `bigint`, not a `number` so you can't use with with lots of math functions. Also, when you convert them to numbers they may lose precision because a `number` can only hold integers of 53 bits in size. So, first we subtract the 2 `bigint` s which stays a `bigint`. Then we convert the result to a number so we can use it as normal.

In the code above, we are are only copying the results to `resultBuffer` some times, when it's not mapped. That means we'll only be reading the time on some frames. Most likely every other frame but there is no strict guarantee how long it will take until `mapAsync` resolves. Because of that, we update `gpuTime` which we can use at anytime to get the last recorded time.

```
    infoElem.textContent = `\
fps: ${(1 / deltaTime).toFixed(1)}
js: ${jsTime.toFixed(1)}ms
gpu: ${canTimestamp ? `${(gpuTime / 1000).toFixed(1)}µs` : 'N/A'}
`;
```

And with that we get a GPU time from WebGPU

click here to open in a separate window

For me, the numbers change too often to see anything useful. One way to fix that is to compute a rolling average. Here's a class to help compute a rolling average.

```
// Note: We disallow negative values as this is used for timestamp queries
// where it's possible for a query to return a beginning time greater than the
// end time. See: https://gpuweb.github.io/gpuweb/#timestamp
class NonNegativeRollingAverage {
  #total = 0;
  #samples = [];
  #cursor = 0;
  #numSamples;
  constructor(numSamples = 30) {
    this.#numSamples = numSamples;
  }
  addSample(v) {
    if (!Number.isNaN(v) && Number.isFinite(v) && v >= 0) {
      this.#total += v - (this.#samples[this.#cursor] || 0);
      this.#samples[this.#cursor] = v;
      this.#cursor = (this.#cursor + 1) % this.#numSamples;
    }
  }
  get() {
    return this.#total / this.#samples.length;
  }
}
```

It keeps an array of values and a total. When a new value is added the oldest value is subtracted from the total as the new value is added.

We can use it like this.

hide deleted

```
const fpsAverage = new NonNegativeRollingAverage();
const jsAverage = new NonNegativeRollingAverage();
const gpuAverage = new NonNegativeRollingAverage();

function render(now) {
  ...

    const commandBuffer = encoder.finish();
    device.queue.submit([commandBuffer]);

    if (canTimestamp && resultBuffer.mapState === 'unmapped') {
      resultBuffer.mapAsync(GPUMapMode.READ).then(() => {
        const times = new BigInt64Array(resultBuffer.getMappedRange());
        gpuTime = Number(times[1] - times[0]);
        gpuAverage.addSample(gpuTime / 1000);
        resultBuffer.unmap();
      });
    }

    const jsTime = performance.now() - startTime;

    fpsAverage.addSample(1 / deltaTime);
    jsAverage.addSample(jsTime);

    infoElem.textContent = `\
fps: ${(1 / deltaTime).toFixed(1)}
js: ${jsTime.toFixed(1)}ms
gpu: ${canTimestamp ? `${(gpuTime / 1000).toFixed(1)}µs` : 'N/A'}
fps: ${fpsAverage.get().toFixed(1)}
js: ${jsAverage.get().toFixed(1)}ms
gpu: ${canTimestamp ? `${gpuAverage.get().toFixed(1)}µs` : 'N/A'}
`;

    requestAnimationFrame(render);
  }
  requestAnimationFrame(render);
}
```

And now the numbers are a little more stable.

click here to open in a separate window

Using a helper

For me, I find all of this a little tedious and probably easy to get something wrong. We had to make 3 things, a querySet and 2 buffers. We had to change our renderPassDescriptor. We had to resolve the results and copy to a mappable buffer.

One way to make this less tedious would be to make a class to helps us do the timing. Here's one example of a helper that might help with some of these issues.

```
function assert(cond, msg = '') {
  if (!cond) {
    throw new Error(msg);
  }
}

// We track command buffers so we can generate an error if
// we try to read the result before the command buffer has been executed.
const s_unsubmittedCommandBuffer = new Set();

/* global GPUQueue */
GPUQueue.prototype.submit = (function(origFn) {
  return function(commandBuffers) {
    origFn.call(this, commandBuffers);
    commandBuffers.forEach(cb => s_unsubmittedCommandBuffer.delete(cb));
  };
})(GPUQueue.prototype.submit);

// See https://webgpufundamentals.org/webgpu/lessons/webgpu-timing.html
export default class TimingHelper {
  #canTimestamp;
```

```
  #device;
  #querySet;
  #resolveBuffer;
  #resultBuffer;
  #commandBuffer;
  #resultBuffers = [];
  // state can be 'free', 'need resolve', 'wait for result'
  #state = 'free';

  constructor(device) {
    this.#device = device;
    this.#canTimestamp = device.features.has('timestamp-query');
    if (this.#canTimestamp) {
      this.#querySet = device.createQuerySet({
         type: 'timestamp',
         count: 2,
      });
      this.#resolveBuffer = device.createBuffer({
        size: this.#querySet.count * 8,
        usage: GPUBufferUsage.QUERY_RESOLVE | GPUBufferUsage.COPY_SRC,
      });
    }
  }

  #beginTimestampPass(encoder, fnName, descriptor) {
    if (this.#canTimestamp) {
      assert(this.#state === 'free', 'state not free');
      this.#state = 'need resolve';

      const pass = encoder[fnName]({
        ...descriptor,
        ...{
          timestampWrites: {
            querySet: this.#querySet,
            beginningOfPassWriteIndex: 0,
            endOfPassWriteIndex: 1,
          },
        },
      });

      const resolve = () => this.#resolveTiming(encoder);
      const trackCommandBuffer = (cb) => this.#trackCommandBuffer(cb);
      pass.end = (function(origFn) {
        return function() {
          origFn.call(this);
          resolve();
        };
      })(pass.end);

      encoder.finish = (function(origFn) {
        return function() {
          const cb = origFn.call(this);
          trackCommandBuffer(cb);
          return cb;
        };
      })(encoder.finish);

      return pass;
    } else {
      return encoder[fnName](descriptor);
    }
  }

  beginRenderPass(encoder, descriptor = {}) {
    return this.#beginTimestampPass(encoder, 'beginRenderPass', descriptor);
  }
```

```
  beginComputePass(encoder, descriptor = {}) {
    return this.#beginTimestampPass(encoder, 'beginComputePass', descriptor);
  }

  #trackCommandBuffer(cb) {
    if (!this.#canTimestamp) {
      return;
    }
    assert(this.#state === 'need finish', 'you must call encoder.finish');
    this.#commandBuffer = cb;
    s_unsubmittedCommandBuffer.add(cb);
    this.#state = 'wait for result';
  }

  #resolveTiming(encoder) {
    if (!this.#canTimestamp) {
      return;
    }
    assert(
      this.#state === 'need resolve',
      'you must use timerHelper.beginComputePass or timerHelper.beginRenderPass',
    );
    this.#state = 'need finish';

    this.#resultBuffer = this.#resultBuffers.pop() || this.#device.createBuffer({
      size: this.#resolveBuffer.size,
      usage: GPUBufferUsage.COPY_DST | GPUBufferUsage.MAP_READ,
    });

    encoder.resolveQuerySet(this.#querySet, 0, this.#querySet.count, this.#resolveBuffer, 0);
    encoder.copyBufferToBuffer(this.#resolveBuffer, 0, this.#resultBuffer, 0, this.#resultBuffer.size);
  }

  async getResult() {
    if (!this.#canTimestamp) {
      return 0;
    }
    assert(
      this.#state === 'wait for result',
      'you must call encoder.finish and submit the command buffer before you can read the result',
    );
    assert(!!this.#commandBuffer); // internal check
    assert(
      !s_unsubmittedCommandBuffer.has(this.#commandBuffer),
      'you must submit the command buffer before you can read the result',
    );
    this.#commandBuffer = undefined;
    this.#state = 'free';

    const resultBuffer = this.#resultBuffer;
    await resultBuffer.mapAsync(GPUMapMode.READ);
    const times = new BigInt64Array(resultBuffer.getMappedRange());
    const duration = Number(times[1] - times[0]);
    resultBuffer.unmap();
    this.#resultBuffers.push(resultBuffer);
    return duration;
  }
}
```

The asserts are there to helps us not use this class wrong. For example if we end a pass but don't resolve it or, if we resolve it and try to read the result but we haven't

submitted.

With this class, we can remove much of the code we had before.

hide deleted

```
async function main() {
  const adapter = await navigator.gpu?.requestAdapter();
  const canTimestamp = adapter.features.has('timestamp-query');
  const device = await adapter?.requestDevice({
    requiredFeatures: [
      ...(canTimestamp ? ['timestamp-query'] : []),
     ],
  });
  if (!device) {
    fail('need a browser that supports WebGPU');
    return;
  }

  const timingHelper = new TimingHelper(device);

  ...

- const { querySet, resolveBuffer, resultBuffer } = (() => {
-   if (!canTimestamp) {
-     return {};
-   }
-
-   const querySet = device.createQuerySet({
-     type: 'timestamp',
-     count: 2,
-   });
-   const resolveBuffer = device.createBuffer({
-     size: querySet.count * 8,
-     usage: GPUBufferUsage.QUERY_RESOLVE | GPUBufferUsage.COPY_SRC,
-   });
-   const resultBuffer = device.createBuffer({
-     size: resolveBuffer.size,
-     usage: GPUBufferUsage.COPY_DST | GPUBufferUsage.MAP_READ,
-   });
-   return {querySet, resolveBuffer, resultBuffer };
- })();

  ...

  function render(now) {

    ...

-   const pass = encoder.beginRenderPass(renderPassDescriptor);
    const pass = timingHelper.beginRenderPass(encoder, renderPassDescriptor);

    ...

    pass.end();

    -if (canTimestamp) {
    -  encoder.resolveQuerySet(querySet, 0, querySet.count, resolveBuffer, 0);
    -  if (resultBuffer.mapState === 'unmapped') {
    -    encoder.copyBufferToBuffer(resolveBuffer, 0, resultBuffer, 0, resultBuffer.size);
    -  }
    -}

    const commandBuffer = encoder.finish();
    device.queue.submit([commandBuffer]);
```

```
timingHelper.getResult().then(gpuTime => {
    gpuAverage.addSample(gpuTime / 1000);
});

...
```

click here to open in a separate window

A few points about the `TimingHelper` class:

- You still have to manually request the `'timestamp-query'` feature when you create your device but, the class handles whether it exists or not on the device.
- When you call `timerHelper.beginRenderPass` or `timerHelper.beginComputePass` it automatically adds the appropriate properties to the pass descriptor. It also returns a pass encoder who's `end` function automatically resolves the queries.
- It's designed so if you use it wrong it will complain.
- It only handles 1 pass.

  There are a bunch of tradeoffs here and without more exploration it's not clear what would be best.

  A class that handles multiple passes could be useful but, ideally, you'd use a single `GPUQuerySet` that has enough space for all of your passes, rather than

1 `GPUQuerySet` per pass.

But, in order to do that you'd either need to have the user tell you up front the maximum number of passes they'll use. Or, you need to make the code more complicated where it starts with a small `GPUQuerySet` and deletes it and makes a new larger one if you use more. But then, at least for 1 frame, you'd need to handle having multiple `GPUQuerySet` s

All of that seemed overkill so for now it seemed best to make it handle one pass and you can build on top of it until you decide it needs to be changed.

You could also make a `NoTimingHelper` .

```
class NoTimingHelper {
  constructor() { }
  beginRenderPass(encoder, descriptor = {}) {
    return encoder.beginTimestampPass(descriptor);
  }

  beginComputePass(encoder, descriptor = {}) {
    return encoder.beginComputePass(descriptor);
  }
  async getResult() { return 0; }
}
```

As one possible way to make so you can add timing and turn it off without having to change too much code.

In any case, I've used the `TimingHelper` class to time the various examples from the articles on using compute shaders to compute image histograms. Here's a list of them. Since only the video example runs continuously it's probably the best example

- 4 channel video histogram

The rest just run once and print their result to the JavaScript console.

- 4 channel workgroup per chunk histogram with reduce
- 4 channel workgroup per pixel histogram
- 4 channel JavaScript histogram
- 1 channel workgroup per chunk histogram with reduce
- 1 channel workgroup per chunk histogram with sum
- 1 channel workgroup per pixel histogram
- 1 channel single core histogram
- 1 channel JavaScript histogram

Important: `timestamp-query` results are implementation-defined

This effectively means you can use them for debugging and for comparing techniques but you can not trust them to return similar results for all of your users. You can't not even assume relative results. Different GPUs work in different ways and are able to optimize rendering and computing across passes. That means on one machine a first pass might take 200µs to draw 100 things and the 2nd pass might also take 200µs to 200 things but, another GPU might take 100µs to draw the first 100 things and 200µs to draw the 2nd 100 things so where as the first GPU had a relative difference of 0µs, the 2nd had a relative difference of 100µs even though both GPUs were asked to draw the same thing.

> By default the `'timestamp-query'` time values are quantized to 100µ seconds. In Chrome, if you enable "enable-webgpu-developer-features" in about:flags, the time values may not be quantized. This would theoretically give you more accurate timings. That said, normally 100µ second quantized values should be enough for you to compare shaders techniques for performance.

---

1. `rAF` is short for `requestAnimationFrame` ↩
2. This only works if the radius of the circle is less than 0.5 but it seemed best not to bloat the code with complicated checks for size. ↩
3. Copying the query results to mappable buffer is only for the purpose of reading the values from JavaScript. If your use-case only needs the results to stay on the GPU, for example as input to something else, then you don't need to copy the results to a mappable buffer. ↩